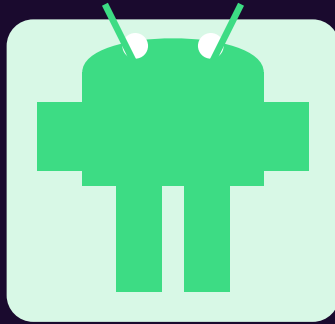




Android Versions App

Documentacion Tecnica — Parcial 2

Autor: Diego Alejandro Groot Cabrera

Profesor: Adolfo Centeno

Entrega: 8 de Mayo de 2026

Flutter | Spring Boot | PostgreSQL | Docker | Render

0 Contents

1	Descripcion General del Proyecto	2
2	Tecnologias Utilizadas	2
3	Patrones de Diseno Utilizados	2
4	Patron Estructural: Facade	3
4.1	Definicion del patron Facade	3
4.2	Estructura general del patron	3
4.3	Por que se uso en este proyecto?	3
4.4	Problema sin Facade vs. solucion con Facade	4
4.5	Codigo implementado: TweetFacade	4
4.6	Codigo implementado: Controladores que usan la Facade	6
4.7	Diagrama UML del patron Facade en el proyecto	7
4.8	Beneficios obtenidos del patron Facade	8
5	Patron Repository	8
5.0.1	Repositorios implementados	8
6	Modelos / Entidades JPA	9
7	Patron MVC	11
8	Endpoints de la API REST	12
9	Arquitectura y Despliegue	12
10	Conclusion	13

1 Descripción General del Proyecto

Resumen: Aplicación multiplataforma para registrar versiones del sistema operativo Android, con soporte de reacciones emoji y comentarios por publicación. Backend REST con Spring Boot y frontend Flutter compilado para web, Linux desktop y APK Android.

La idea del proyecto es tener una especie de red social enfocada en versiones de Android. Cada usuario puede crear una publicación con el nombre de la versión, su fecha, descripción y características. Otros usuarios pueden reaccionar con emojis (**like**, **love**, **haha**, **wow**, **sad**) y también dejar comentarios de texto.

El backend se hizo con **Spring Boot** y expone una API REST. La base de datos es **PostgreSQL** en producción (Aiven). El frontend se construyó con **Flutter**, compilando la misma base de código para web y APK de Android. El proyecto está desplegado en **Render** usando **Docker**.

2 Tecnologías Utilizadas

Tecnología	Uso en el proyecto
Flutter (Dart)	Interfaz: web, Linux desktop y APK Android
Spring Boot 4	Framework backend, expone la API REST
PostgreSQL 16	Base de datos relacional en producción y local
Hibernate / JPA	Mapeo objeto-relacional de las entidades
Docker	Contenedorización del backend para despliegue
Render	Plataforma de despliegue en la nube
Maven	Gestión de dependencias y build del proyecto Java
GitHub	Control de versiones

Table 1: Stack tecnológico del proyecto

3 Patrones de Diseño Utilizados

En el proyecto se utilizaron tres patrones de diseño de software: **Facade** (estructural), **Repository** (arquitectural) y **MVC** (arquitectural). El patrón más importante desde la perspectiva de diseño estructural es Facade, que se analiza en profundidad en la siguiente sección.

4 Patron Estructural: Facade

¿Que es un patron estructural?

Los **patrones estructurales** se ocupan de como se componen las clases y los objetos para formar estructuras mas grandes. No se enfocan en crear objetos (eso es trabajo de los patrones creacionales) ni en como interactuan en tiempo de ejecucion (eso es trabajo de los comportamentales). Su objetivo es simplificar las relaciones entre clases, reducir el acoplamiento y hacer que el sistema sea mas facil de entender y mantener. Ejemplos de patrones estructurales del catalogo GoF: **Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy**.

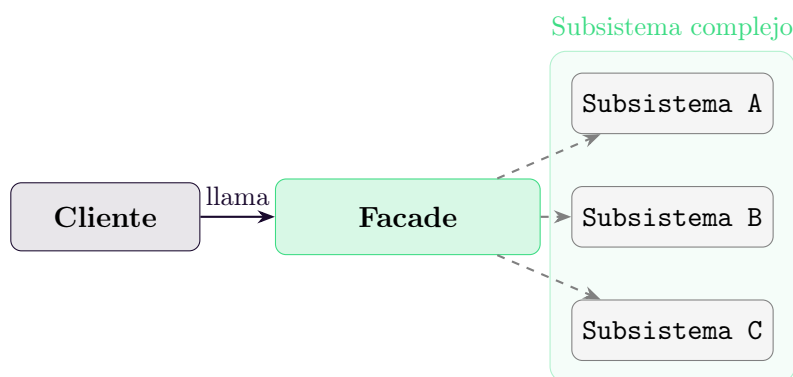
4.1 Definicion del patron Facade

¿Que es el patron Facade?

Facade (Fachada) proporciona una interfaz simplificada a un conjunto de interfaces en un subsistema. Define una interfaz de nivel superior que hace que el subsistema sea mas facil de usar. En lugar de que el cliente tenga que conocer y coordinar multiples clases internas, habla con una sola clase fachada que lo resuelve todo internamente.

La analogia clasica es la de un cajero automatico: el usuario solo presiona botones, sin saber que por detras hay comunicacion con el banco, verificacion de saldo, actualizacion de registros contables y emision de comprobantes. Toda esa complejidad esta *detras de la fachada*.

4.2 Estructura general del patron



4.3 Por que se uso en este proyecto?

Sin un Facade, cada controlador REST tendria que importar directamente **AndroidRepository**, **ComentarioRepository** y **ReaccionRepository**, y ademas conocer la logica de como buscar un tweet, validar que existe, crear la entidad relacionada y guardarla. Eso genera **acoplamiento fuerte**: si cambia la logica de almacenamiento, hay que modificar cada controlador.

Con `TweetFacade`, los controladores solo conocen a la fachada y le delegan todo. La fachada *esconde* la complejidad de coordinar tres repositorios.

4.4 Problema sin Facade vs. solución con Facade

Sin Facade: controlador acoplado a múltiples repositorios

```
1 // SIN FACADE - el controlador tiene que conocer TODOS los
  repositorios
2 @RestController
3 public class ComentarioController {
4     // Acoplado a 3 repositorios directamente
5     private final AndroidRepository tweetRepo;
6     private final ComentarioRepository comentarioRepo;
7     private final ReaccionRepository reaccionRepo;
8
9     @PostMapping("/{id}/comentarios")
10    public ResponseEntity<?> agregar(@PathVariable Long id,
11                                     @RequestBody Map<String,
12                                     String> body) {
13        // Logica duplicada en cada controlador
14        android_tweet tweet = tweetRepo.findById(id)
15            .orElseThrow(() -> new RuntimeException("No existe
16            "));
17        Comentario c = new Comentario(body.get("texto"), tweet
18    );
19        comentarioRepo.save(c);
20        return ResponseEntity.ok(c);
21    }
22 }
```

Con Facade: controlador simple y lógica centralizada

```
1 // CON FACADE - el controlador solo conoce a TweetFacade
2 @RestController
3 public class ComentarioController {
4     private final TweetFacade tweetFacade; // UNA sola
5     dependencia
6
7     @PostMapping("/{id}/comentarios")
8     public ResponseEntity<?> agregar(@PathVariable Long id,
9                                     @RequestBody Map<String,
10                                     String> body) {
11        // Solo delega, no sabe como funciona internamente
12        Comentario c = tweetFacade.agregarComentario(id, body.
13        get("texto"));
14        return ResponseEntity.ok(c);
15    }
16 }
```

4.5 Código implementado: `TweetFacade`

Esta es la clase fachada real del proyecto. Coordina los tres repositorios internamente y expone métodos simples y cohesivos:

TweetFacade.java -- La Fachada

```
1  @Service
2  public class TweetFacade {
3
4      // Los tres subsistemas que la fachada coordina
5      private final AndroidRepository tweetRepository;
6      private final ComentarioRepository comentarioRepository;
7      private final ReaccionRepository reaccionRepository;
8
9      public TweetFacade(AndroidRepository tweetRepository,
10                          ComentarioRepository comentarioRepository,
11                          ReaccionRepository reaccionRepository) {
12          this.tweetRepository = tweetRepository;
13          this.comentarioRepository = comentarioRepository;
14          this.reaccionRepository = reaccionRepository;
15      }
16
17      // Operacion 1: agregar comentario (busca tweet + crea + guarda)
18      public Comentario agregarComentario(Long tweetId, String
19      texto) {
20          android_tweet tweet = tweetRepository.findById(tweetId)
21              .orElseThrow(() -> new RuntimeException(
22                  "Error: El Tweet con ID " + tweetId
23                  + " no existe en la DB."));
24          Comentario comentario = new Comentario(texto, tweet);
25          return comentarioRepository.save(comentario);
26      }
27
28      // Operacion 2: listar comentarios de un tweet
29      public List<Comentario> obtenerComentarios(Long tweetId) {
30          return comentarioRepository.findByTweetId(tweetId);
31      }
32
33      // Operacion 3: agregar reaccion (busca tweet + crea + guarda)
34      public Reaccion agregarReaccion(Long tweetId, String tipo) {
35          android_tweet tweet = tweetRepository.findById(tweetId)
36              .orElseThrow(() -> new RuntimeException(
37                  "Tweet no encontrado"));
38          Reaccion reaccion = new Reaccion(tipo, tweet);
39          return reaccionRepository.save(reaccion);
40      }
41
42      // Operacion 4: listar reacciones
43      public List<Reaccion> obtenerReacciones(Long tweetId) {
44          return reaccionRepository.findByTweetId(tweetId);
45      }
46
47      // Operacion 5: contar reacciones por tipo (like:5, love:2, ...)
48      public Map<String, Long> contarReacciones(Long tweetId) {
49          List<Reaccion> reacciones =
50              reaccionRepository.findByTweetId(tweetId);
51          Map<String, Long> conteo = new java.util.HashMap<>();
52          for (Reaccion r : reacciones) {
```

```
52         conteo.merge(r.getTipo(), 1L, Long::sum);
53     }
54     return conteo;
55 }
56 }
```

4.6 Código implementado: Controladores que usan la Facade

ComentarioController.java

```
1  @CrossOrigin(origins = "*")
2  @RestController
3  @RequestMapping("/api/tweets")
4  public class ComentarioController {
5
6      private final TweetFacade tweetFacade;
7
8      public ComentarioController(TweetFacade tweetFacade) {
9          this.tweetFacade = tweetFacade;
10     }
11
12     @PostMapping("/{id}/comentarios")
13     public ResponseEntity<?> agregar(
14         @PathVariable Long id,
15         @RequestBody Map<String, String> body) {
16         try {
17             Comentario c = tweetFacade.agregarComentario(
18                 id, body.get("texto"));
19             return ResponseEntity.ok(c);
20         } catch (RuntimeException e) {
21             return ResponseEntity.status(404)
22                 .body(Map.of("error", e.getMessage()));
23         }
24     }
25
26     @GetMapping("/{id}/comentarios")
27     public ResponseEntity<List<Comentario>> listar(
28         @PathVariable Long id) {
29         return ResponseEntity.ok(
30             tweetFacade.obtenerComentarios(id));
31     }
32 }
```

ReaccionController.java

```
1  @CrossOrigin(origins = "*")
2  @RestController
3  @RequestMapping("/api/tweets")
4  public class ReaccionController {
5
6      private final TweetFacade tweetFacade;
7
8      public ReaccionController(TweetFacade tweetFacade) {
9          this.tweetFacade = tweetFacade;
10     }
```

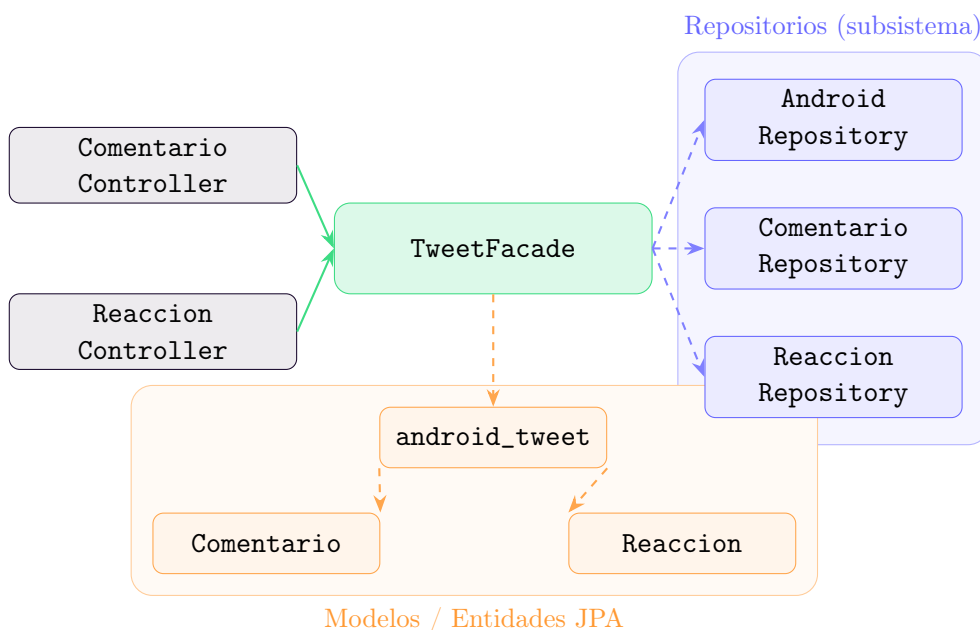
```

11
12     @PostMapping("/{id}/reacciones")
13     public ResponseEntity<?> reaccionar(
14         @PathVariable Long id,
15         @RequestBody Map<String, String> body) {
16         try {
17             Reaccion r = tweetFacade.agregarReaccion(
18                 id, body.get("tipo"));
19             return ResponseEntity.ok(r);
20         } catch (RuntimeException e) {
21             return ResponseEntity.status(404)
22                 .body(Map.of("error", e.getMessage()));
23         }
24     }
25
26     @GetMapping("/{id}/reacciones")
27     public ResponseEntity<List<Reaccion>> listar(
28         @PathVariable Long id) {
29         return ResponseEntity.ok(
30             tweetFacade.obtenerReacciones(id));
31     }
32
33     @GetMapping("/{id}/reacciones/conteo")
34     public ResponseEntity<Map<String, Long>> conteo(
35         @PathVariable Long id) {
36         return ResponseEntity.ok(
37             tweetFacade.contarReacciones(id));
38     }
39 }

```

4.7 Diagrama UML del patron Facade en el proyecto

El siguiente diagrama muestra como TweetFacade actua como la unica puerta de entrada que los controladores usan para acceder al subsistema de repositorios y modelos:



4.8 Beneficios obtenidos del patron Facade

Beneficio	Impacto en el proyecto
Bajo acoplamiento	Los controladores no dependen de los repositorios directamente
Unico punto de cambio	Si cambia la logica de comentarios, solo se modifica TweetFacade
Codigo limpio	Cada controlador tiene menos de 30 lineas de codigo
Logica centralizada	El conteo de reacciones por tipo vive en un solo lugar
Facil de testear	Se puede hacer mock de TweetFacade sin levantar repositorios

Table 2: Beneficios del patron Facade en el proyecto

5 Patron Repository

¿Que es el patron Repository?

El Repository actua como una coleccion de objetos en memoria. En lugar de escribir SQL directamente en la logica de negocio, se crea una interfaz que define que operaciones se pueden hacer con cada entidad. Spring Data JPA genera automaticamente la implementacion a partir del nombre de los metodos, sin necesidad de escribir SQL.

Repositorios implementados

ComentarioRepository.java

```

1 @Repository
2 public interface ComentarioRepository
3     extends JpaRepository<Comentario, Long> {
4
5     // Spring genera automaticamente:
6     // SELECT * FROM comentarios WHERE tweet_id = ?
7     List<Comentario> findByTweetId(Long tweetId);
8 }

```

ReaccionRepository.java

```

1 @Repository
2 public interface ReaccionRepository
3     extends JpaRepository<Reaccion, Long> {
4
5     // SELECT * FROM reacciones WHERE tweet_id = ?
6     List<Reaccion> findByTweetId(Long tweetId);

```

```

7
8     // SELECT COUNT(*) FROM reacciones WHERE tweet_id = ?
9     long countByTweetId(Long tweetId);
10 }

```

AndroidRepository.java

```

1 @Repository
2 public interface AndroidRepository
3     extends JpaRepository<android_tweet, Long> {
4
5     // SELECT * FROM android_versions WHERE user_id = ?
6     List<android_tweet> findByUserId(Long userId);
7 }

```

UserRepository.java

```

1 @Repository
2 public interface UserRepository
3     extends JpaRepository<User, Long> {
4
5     // SELECT * FROM users WHERE correo = ?
6     Optional<User> findByCorreo(String correo);
7
8     // SELECT COUNT(*) > 0 FROM users WHERE correo = ?
9     boolean existsByCorreo(String correo);
10 }

```

6 Modelos / Entidades JPA

Los modelos son clases Java anotadas con `@Entity` que Hibernate mapea automáticamente a tablas de PostgreSQL.

android_tweet.java -- Entidad principal

```

1 @Entity
2 @Table(name = "android_versions")
3 public class android_tweet {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @NotBlank @Size(max = 100)
10    private String nombre;
11
12    @NotBlank
13    private String fecha;
14
15    @NotBlank @Size(max = 500)
16    private String descripcion;
17

```

```
18     @Column(length = 1000)
19     private String características;
20
21     @Column(name = "url_photo")
22     private String urlPhoto;
23
24     // Relacion: muchas versiones pertenecen a un usuario
25     @ManyToOne(fetch = FetchType.LAZY)
26     @JoinColumn(name = "user_id")
27     @JsonIgnore
28     private User user;
29 }
```

Comentario.java

```
1  @Entity
2  @Table(name = "comentarios")
3  public class Comentario {
4
5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Long id;
8
9      private String texto;
10
11     // Relacion: muchos comentarios pertenecen a un tweet
12     @ManyToOne(fetch = FetchType.LAZY)
13     @JoinColumn(name = "tweet_id")
14     @JsonIgnore // Evita referencia circular en la serializacion
15     // JSON
16     private android_tweet tweet;
17 }
```

Reaccion.java

```
1  @Entity
2  @Table(name = "reacciones")
3  public class Reaccion {
4
5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Long id;
8
9      private String tipo; // "like", "love", "haha", "wow", "sad"
10
11     // Relacion: muchas reacciones pertenecen a un tweet
12     @ManyToOne(fetch = FetchType.LAZY)
13     @JoinColumn(name = "tweet_id")
14     @JsonIgnore
15     private android_tweet tweet;
16 }
```

User.java

```
1 @Entity
2 @Table(name = "users")
3 public class User {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @NotBlank
10    private String nombre;
11
12    @NotBlank
13    @Column(unique = true)
14    private String correo;
15
16    @NotBlank
17    private String contrasena;
18
19    // Un usuario puede tener muchas versiones de Android
20    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
21    @JsonIgnore
22    private List<android_tweet> versiones = new ArrayList<>();
23 }
```

7 Patron MVC

¿Que es el patron MVC?

MVC divide la aplicacion en tres capas: el **Modelo** maneja los datos, la **Vista** muestra la interfaz al usuario, y el **Controlador** recibe las peticiones y coordina entre los otros dos. Esto permite cambiar la interfaz sin tocar la logica del negocio.

En este proyecto MVC se aplica en dos niveles simultaneos:

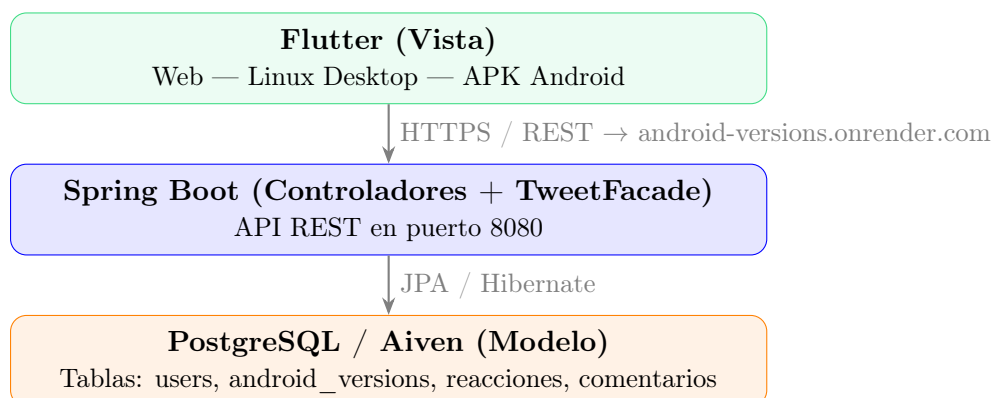
- **Backend (Spring Boot):** las entidades JPA son el Modelo, los `@RestController` son los Controladores, y el JSON que retorna la API es la representacion de la Vista.
- **Frontend (Flutter):** los widgets de Flutter son la Vista que consume la API y muestra los datos al usuario.

8 Endpoints de la API REST

Metodo	Ruta	Descripcion
Versiones de Android		
GET	/api/android-versions?userId={id}	Obtener versiones de un usuario
POST	/api/android-versions?userId={id}	Crear nueva version
DELETE	/api/android-versions/{id}	Eliminar version por ID
Usuarios		
POST	/api/users	Registrar nuevo usuario
POST	/api/users/login	Iniciar sesion
PUT	/api/users/{id}	Actualizar datos del usuario
DELETE	/api/users/{id}	Eliminar cuenta
Reacciones y Comentarios (via Facade)		
POST	/api/tweets/{id}/reacciones	Agregar reaccion
GET	/api/tweets/{id}/reacciones/conteo	Conteo por tipo
POST	/api/tweets/{id}/comentarios	Agregar comentario
GET	/api/tweets/{id}/comentarios	Lista de comentarios

Table 3: Endpoints completos de la API REST

9 Arquitectura y Despliegue



El backend se empaqueta con Maven en un JAR y se contenedoriza con Docker. Las variables de entorno `DATABASE_URL`, `DB_USER` y `DB_PASSWORD` configuran la conexión a Aiven desde Render.

```
# Desarrollo local
$ mvn spring-boot:run -Dspring-boot.run.profiles=local

# Produccion (Render lo ejecuta automaticamente via Docker)
$ flutter build web -release    # frontend web
$ flutter build apk -release    # APK Android
```

10 Conclusion

El patron **Facade** fue el aporte estructural mas importante del proyecto. Al encapsular toda la coordinacion entre `AndroidRepository`, `ComentarioRepository` y `ReaccionRepository` dentro de `TweetFacade`, se logro que los controladores REST quedaran con una sola dependencia y menos de 30 lineas de codigo cada uno. Eso hace el sistema mas facil de mantener: si la logica de conteo de reacciones cambia, solo se modifica la fachada.

Los **repositorios** eliminaron la necesidad de escribir SQL, ya que Spring Data JPA genera las consultas automaticamente a partir del nombre de los metodos como `findByTweetId` o `existsByCorreo`.

El patron **MVC** permitio desarrollar el backend (Spring Boot) y el frontend (Flutter) de forma completamente independiente, comunicandose unicamente a traves de la API REST desplegada en Render.